

C++17 Interview Cheat Sheet

vector

```
#include <vector>
vector<int> v, v2(10), v3(10, 5);
v.push_back(10); v.emplace_back(20);
v.pop_back();
v.size(); v.empty(); v.front(); v.back();
v[0]; v.at(0); v.begin(); v.end();
sort(v.begin(), v.end()); reverse(v.begin(),
v.end());
v.clear();
```

string

```
#include <string>
string s = "hello";
s.size(); s.length(); s.empty();
s.push_back('a'); s.pop_back();
s.substr(pos, len); s.find("abc");
s.erase(pos, len); s.insert(pos, "abc");
reverse(s.begin(), s.end()); sort(s.begin(),
s.end());
stoi(s); stol(s); stoll(s); to_string(123); s +=
"abc";

// string_view (C++17, non-owning view)
#include <string_view>
string_view sv = s;
sv.remove_prefix(n); sv.remove_suffix(n);
sv.substr(pos, len); // 0(1)
```

maps (unordered / ordered)

```
#include <unordered_map> // unordered_map
#include <map> // map, multimap
// unordered_map<int,int> mp; // 0(1) avg
// map<int,int> mp; // 0(log n)
// multimap<int,int> mmp; // sorted,
duplicate keys
mp[key]++; mp[key] = value;
mp.emplace(key, value); mp.count(key);
mp.contains(key); mp.find(key); mp.erase(key);
// map only:
mp.lower_bound(key); mp.upper_bound(key);
// multimap: equal_range(key)
for(auto &[k,v] : mp) { }
```

sets (unordered / ordered)

```
#include <unordered_set> // unordered_set
#include <set> // set, multiset
// unordered_set<int> st; // 0(1) avg
// set<int> st; // 0(log n)
// multiset<int> mst; // sorted, duplicate
values
st.insert(x); st.emplace(x); st.erase(x);
st.count(x); st.contains(x); st.find(x);
// set only:
st.lower_bound(x); st.upper_bound(x);
// multiset: equal_range(x)
```

queues & stacks

```
#include <queue> // queue, priority_queue
#include <deque> // deque
#include <stack> // stack
// queue<int> q; // or deque<int> dq;
q.push(x); q.emplace(x); q.pop(); q.front();
```

```
q.back();
// deque only:
dq.push_front(x); dq.pop_front();

// stack<int> st;
st.push(x); st.emplace(x); st.pop(); st.top();
st.empty();
```

priority_queue (Heaps)

```
#include <queue>
// Max Heap (default)
priority_queue<int> pq_max;
// Min Heap
priority_queue<int, vector<int>, greater<int>>
pq_min;

pq.push(x); pq.emplace(x); pq.pop(); pq.top();
pq.empty();
```

pairs & tuples

```
#include <utility> // pair
#include <tuple> // tuple
pair<int,int> p = {1,2}; // or make_pair(1,2)
p.first; p.second;

tuple<int,int,string> t = make_tuple(1,2,"abc");
get<0>(t); get<1>(t); get<2>(t);
auto [x,y,z] = t; // structured binding
```

optional (C++17)

```
#include <optional>
optional<int> o = 42; // or nullopt
o.has_value(); o.value(); o.value_or(-1);
*o; if(o) { }
```

algorithms

```
#include <algorithm>
// Sorting & Searching
sort(v.begin(), v.end()); // rbegin/rend for
desc
reverse(v.begin(), v.end());
nth_element(v.begin(), v.begin()+k, v.end()); //
0(n) partial sort
binary_search(v.begin(), v.end(), x);
lower_bound(v.begin(), v.end(), x);
upper_bound(v.begin(), v.end(), x);
find(v.begin(), v.end(), x); count(v.begin(),
v.end(), x);
min_element(v.begin(), v.end());
max_element(v.begin(), v.end());
next_permutation(v.begin(), v.end());
unique(v.begin(), v.end());
erase(remove(v.begin(), v.end(), x),
v.end()); // erase-remove

// Functional
transform(v.begin(), v.end(), v2.begin(), [](int
x){ return x*2; });
```

numeric & utility

```
#include <numeric> // accumulate, gcd, lcm,
iota
#include <iterator> // distance, advance
// Math
min(a,b); max(a,b); swap(a,b); abs(x); clamp(x,
lo, hi);
```

```
accumulate(v.begin(), v.end(), 0);
gcd(a,b); lcm(a,b); iota(v.begin(), v.end(), 0);

// Iterators
auto it = find(v.begin(), v.end(), x);
distance(v.begin(), it); advance(it, n);
```

bitset & stringstream

```
#include <bitset> // bitset
#include <sstream> // stringstream
bitset<32> b(x);
b.count(); b.test(i); b.set(i); b.reset(i);
b.flip(i);

stringstream ss(s); string word;
while(ss >> word) { }
```

lambda & loops

```
// Lambda
sort(v.begin(), v.end(), [](auto &a, auto &b) {
return a.second < b.second;
});

// Range loops
for(auto x : v) { }
for(auto &x : v) { }
for(auto &[k,v] : mp) { }

// Structured binding (anywhere)
auto [a,b] = p;
auto [x,y,z] = t;
```

Complexity Cheat

Container	Find	Insert	Erase
vector	O(n)	O(1)	O(n)
unord. map/set	O(1)	O(1)	O(1)
map / set	O(log n)	O(log n)	O(log n)
priority_queue	top O(1)	O(log n)	O(log n)
queue / stack	O(1)	O(1)	O(1)

Top 15 Interview Tools

Most Common:

1. vector, 2. unordered_map, 3. unordered_set, 4. string, 5. queue (BFS),
6. stack, 7. priority_queue, 8. pair, 9. sort(), 10. lower_bound(),
11. accumulate(), 12. reverse(), 13. erase(remove()), 14. lambda, 15. deque